NERD - BD – Etapa 3

JOIN e execução de consultas no modelo materializado: Nested Loop Join, Hash Join e GROUP BY

Rafael Lima Dias
Projeto de extensão NERD - BD (Banco de Dados)

Novembro de 2026

Resumo

Este relatório descreve a etapa 3 do motor de consultas didático do Grupo A, escrito em C11 e baseado no modelo de execução materializado. A etapa acrescenta o suporte a duas tabelas (`funcionarios` e `departamentos`), o Nested Loop Join, o Hash Join com fases de BUILD e PROBE explícitas, o GROUP BY por hash e a instrumentação de cada operador. A consulta completa do enunciado foi medida com os dois algoritmos de JOIN, variando a cardinalidade (de 10 mil a 10 milhões de funcionários), a seletividade do filtro (1, 10, 50 e 90%) e o tamanho da relação de BUILD (de 100 a 100 mil departamentos), com 5 repetições por configuração e mediana. Com 1 milhão de funcionários, a consulta levou 254,5 ms com Hash Join e 1 821,7 ms com Nested Loop; com 10 milhões, 2,7 s e 19,9 s. A validação com Valgrind terminou sem vazamentos e sem erros. O relatório traz ainda a arquitetura, as decisões de implementação, o gerenciamento de memória, as dificuldades, as limitações, as otimizações possíveis, o material de extensão e a resposta à questão da etapa.

1 Introdução

Na etapa 2, o motor executava o pipeline SCAN → FILTER → PROJECT → AGGREGATE sobre uma única tabela. A etapa 3 introduz uma segunda tabela e o operador de JOIN, que é o ponto em que o algoritmo escolhido mais pesa no custo da consulta [1, 2]. No modelo materializado, cada operador consome as entradas inteiras e produz uma tabela completa e independente antes que o operador seguinte comece. A consulta de referência é

```
SELECT d.nome, AVG(f.salario)
FROM funcionarios f
JOIN departamentos d ON f.departamento_id = d.id
WHERE f.salario > 5000
GROUP BY d.nome;
```

e é executada pelo plano

```
SCAN funcionarios  -> FILTER salary > 5000 --+
                                             +--> JOIN f.department_id = d.id
SCAN departamentos --------------------------+
                                                  |
                                                  v
                                        PROJECT d.name, f.salary
                                                  |
                                                  v
                                        GROUP BY d.name / AVG(f.salary)
```

O código segue o padrão do repositório, com nomes em inglês: `departamento_id`, `salario`, `idade`, `nome` e `regiao` se chamam `department_id`, `salary`, `age`, `name` e `region`. Os CSV são lidos pela ordem das colunas do esquema.

Escopo. No repositório, cada modelo de execução é de um grupo: Grupo A (Materialização), Grupo B (Pipeline) e Grupo C (Vetorizado). Este trabalho implementa e mede o modelo materializado.

As comparações entre modelos (seção 9 do enunciado) e o teste de tamanho de batch (seção 8.4, a cargo do modelo vetorizado) dependem das implementações dos outros grupos e não foram medidas aqui. Quando o texto trata de pipeline ou vetorizado, trata-se de análise do comportamento esperado, e isso é dito explicitamente.

A Tabela 1 relaciona as entregas da seção 11 do enunciado com as partes deste relatório.

Item do enunciado	Onde está
11.1 Código	`src/join.c`, `src/group_by.c`, `src/table.c`, `src/main.c`, `include/`, `tests/`; Seções 2 a 7
11.2 Experimentos	Seção 8; dados brutos em `results/raw/`, gráficos em `docs/img/`
11.3 Documentação técnica	Seções 2 a 5 e 10 a 12
11.4 Material de extensão	Seção 13
Questão da seção 10	Seção 9

Tabela 1: Entregas da etapa 3.

2 Arquitetura

2.1 Módulos

A Tabela 2 resume a organização do código nesta etapa.

Arquivo	Responsabilidade
`include/table.h`, `src/table.c`	`Schema`, `Value`, `Row` (tupla de montagem), `Table` compacta, leitura de CSV tipada, `value_equals` e `value_hash`
`include/materialized.h`	API dos operadores, `QueryMetrics` e `JoinMetrics`
`src/scan.c`	SCAN, com reserva da capacidade exata
`src/materialized.c`	FILTER e PROJECT
`src/aggregate.c`	AGGREGATE sem agrupamento
`src/join.c`	Nested Loop Join, Hash Join (BUILD e PROBE) e impressão das métricas
`src/group_by.c`	GROUP BY com índice hash de grupos
`src/main.c`	`miniquery`: consulta completa com os dois JOINs; datasets opcionais pela linha de comando
`bench/benchmark.c`	`miniquery_bench`: consulta instrumentada, uma linha CSV por repetição
`bench/generate_data.py`	Gerador determinístico de `funcionarios` e `departamentos`
`bench/run_experiments.py`	Gera os datasets, roda os experimentos e grava `results/raw/`
`bench/summarize.py`	Medianas, tabelas (`results/tabelas.md`) e gráficos (`docs/img/`)
`tests/test_materialized.c`	Testes das etapas 2 e 3
`Makefile`	Alvos `all`, `test`, `sanitize`, `valgrind`, `bench`, `experiments`, `clean`

Tabela 2: Módulos da etapa 3.

2.2 Fluxograma

A Figura 1 mostra o fluxo da consulta completa, com os dois SCANs, o FILTER, o JOIN (com o BUILD e o PROBE do Hash Join), o PROJECT, o GROUP BY e a ordem de liberação das tabelas.

2.3 Suporte a múltiplas tabelas

`funcionarios` e `departamentos` têm esquemas próprios e coexistem em memória como duas `Table` independentes (Tabela 3).

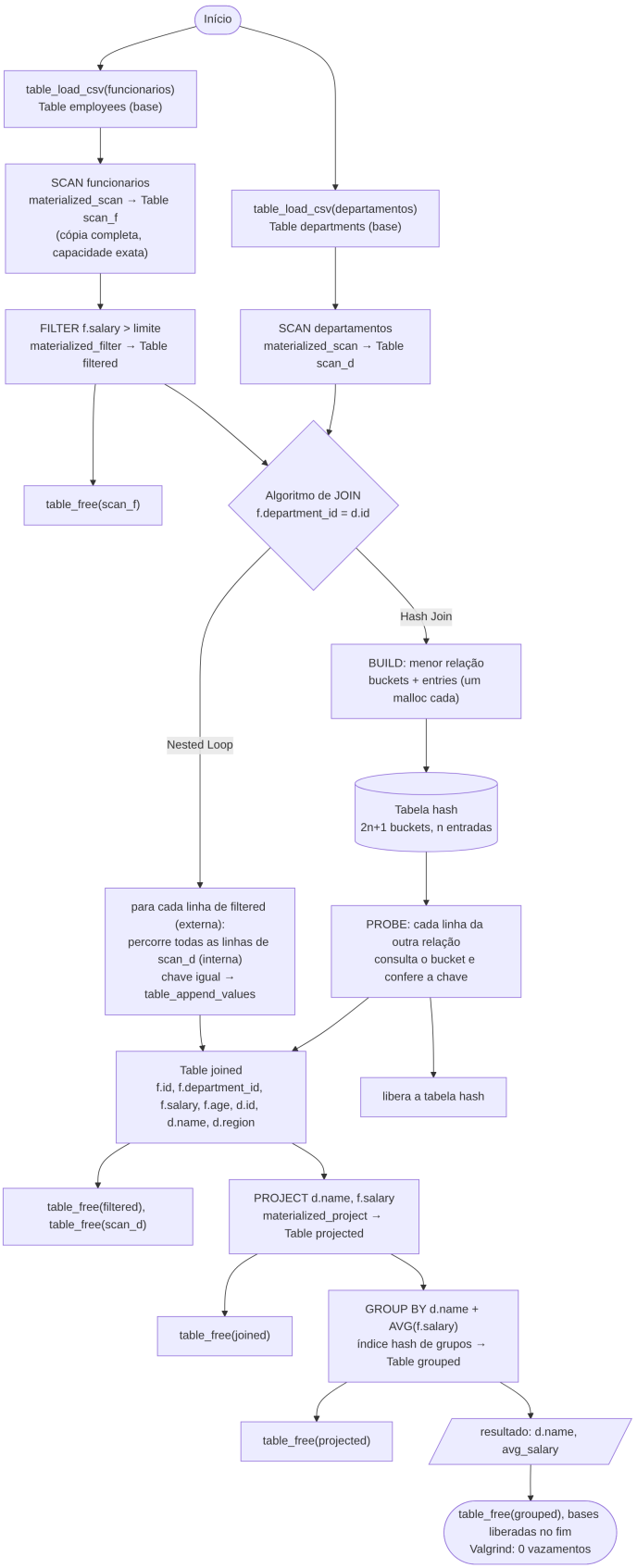


Figura 1: Fluxograma da consulta completa no modelo materializado.

Tabela	Esquema	Bytes por linha
`funcionarios`	`id INT, department_id INT, salary DOUBLE, age INT`	$4 \times 32 = 128$
`departamentos`	`id INT, name STRING, region STRING`	$3 \times 32 = 96$
saída do JOIN	`f.id, f.department_id, f.salary, f.age, d.id, d.name, d.region`	$7 \times 32 = 224$
saída do PROJECT	`d.name, f.salary`	$2 \times 32 = 64$

Tabela 3: Esquemas das tabelas da consulta.

Os pontos pedidos na seção 2 do enunciado ficam assim:

- **Entradas do JOIN:** duas `const Table *`, a esquerda e a direita, com o nome da coluna de chave de cada uma e um alias (`"f"` e `"d"`). As entradas não são alteradas.
- **Esquema resultante:** `join_schema` concatena as colunas da esquerda e da direita prefixadas pelo alias (`f.id`, ..., `d.region`). Nomes repetidos, como `id`, ficam inequívocos, e a ordem não depende de qual lado foi usado como BUILD.
- **Tuplas resultantes:** cada par com chaves iguais vira uma linha da saída, com os valores da esquerda seguidos dos da direita (`append_joined_row`). É um INNER JOIN: linhas sem par não aparecem e chaves repetidas geram todas as combinações.
- **Alocação e liberação:** o JOIN aloca a tabela de saída e, no Hash Join, a tabela hash. A tabela hash é liberada pelo próprio JOIN antes de retornar; a saída pertence ao chamador. Em erro, o JOIN libera a saída parcial e a tabela hash.

3 Funcionamento dos operadores

3.1 SCAN, FILTER, PROJECT e AGGREGATE

Os operadores da etapa 2 mantêm o contrato: entrada `const`, saída nova, retorno `1` ou `0`, saída parcial liberada em erro e métricas opcionais. Nesta etapa, o SCAN e o PROJECT passaram a reservar a capacidade exata (`input->size`) de uma vez, e todos leem e gravam pela representação compacta (Seção 4.1).

3.2 Nested Loop Join

`materialized_nested_loop_join` usa a entrada esquerda como relação externa e a direita como relação interna. As duas já chegam materializadas. Para cada linha externa, o operador percorre todas as linhas internas e compara as chaves com `value_equals`:

```
for (i = 0; i < left->size; ++i) {
    const Value *outer = table_row_const(left, i);
    for (j = 0; j < right->size; ++j) {
        const Value *inner = table_row_const(right, j);
        if (value_equals(&outer[left_index], &inner[right_index]) &&
            !append_joined_row(output, left, outer, right, inner)) {
            table_free(output);
            return 0;
        }
    }
    comparisons += right->size;
}
```

O custo é de $|R| \times |S|$ comparações, e a memória adicional é só a da saída. Não há fase de construção, então `build_time_ms` vale zero. Nas métricas, `build_rows` é a relação interna (percorrida de novo a cada linha externa) e `probe_rows` é a externa. Na consulta, a externa é `funcionarios` filtrado e a interna é `departamentos`.

3.3 Hash Join: BUILD e PROBE

`materialized_hash_join` escolhe como BUILD a menor relação (a esquerda, em caso de empate) e usa a outra como PROBE. A tabela hash é formada por dois vetores alocados de uma vez: `heads`, com

$2n + 1$ buckets, e `entries`, com uma entrada `{row_index, next}` por linha de BUILD, encadeadas por índice. Ela ocupa $(2n + 1) \times 8 + n \times 16$ bytes.

```
/* BUILD: one bucket array plus one entry per BUILD row, both allocated up front. */
table->bucket_count = build->size * 2 + 1;
table->heads = malloc(table->bucket_count * sizeof(*table->heads));
table->entries = malloc(build->size * sizeof(*table->entries));
for (i = 0; i < table->bucket_count; ++i) table->heads[i] = HASH_CHAIN_END;
for (i = 0; i < build->size; ++i) {
    size_t bucket = value_hash(table_cell(build, i, key)) % table->bucket_count;
    table->entries[i].row_index = i;
    table->entries[i].next = table->heads[bucket];
    table->heads[bucket] = i;
}
```

No PROBE, cada linha calcula o hash da sua chave, percorre a cadeia do bucket, confere a igualdade (a cadeia mistura colisões e chaves repetidas) e grava a linha combinada na ordem `f` seguida de `d`:

```
for (i = 0; i < probe->size && hash.bucket_count != 0; ++i) {
    const Value *probe_row = table_row_const(probe, i);
    const Value *probe_value = &probe_row[probe_key];
    size_t entry = hash.heads[value_hash(probe_value) % hash.bucket_count];
    while (entry != HASH_CHAIN_END) {
        const Value *build_row = table_row_const(build, hash.entries[entry].row_index);
        ++comparisons;
        if (value_equals(&build_row[build_key], probe_value)) {
            /* append f.* followed by d.* to the output table */
        }
        entry = hash.entries[entry].next;
    }
}
```

O tempo esperado é $O(|R|+|S|+\text{resultado})$, e a memória adicional é $O(\min(|R|, |S|))$ para a tabela hash, mais a saída. A tabela hash é liberada antes de o operador retornar, e as métricas guardam o seu tamanho.

A Tabela 4 responde aos pontos da seção 4 do enunciado sobre o Hash Join em cada modelo.

Pergunta	Modelo materializado (medido)	Pipeline e vetorizado (esperado, não medido)
Que partes podem funcionar em pipeline?	Nenhuma: cada operador termina antes do próximo. O PROBE processa linha a linha, mas grava tudo numa tabela	O PROBE e o que vem depois (PROJECT, acumulação do GROUP BY) podem consumir as linhas à medida que são produzidas
Que etapas precisam esperar?	Todas esperam a anterior. O BUILD espera o SCAN de departamentos; o PROBE espera o BUILD e o FILTER	O BUILD precisa consumir a relação inteira antes do primeiro PROBE, em qualquer modelo. O GROUP BY só emite no fim
Que estruturas intermediárias são mantidas?	Tabelas de SCAN, FILTER, JOIN e PROJECT, mais a tabela hash durante o JOIN	Só a tabela hash e o estado dos grupos; no vetorizado, também um lote por operador
Que etapas consomem mais memória?	A saída do JOIN (224 bytes por linha) e a do FILTER, vivas ao mesmo tempo. A tabela hash é pequena: 31 KiB com 1 000 departamentos e 3,1 MiB com 100 000	A tabela hash do BUILD passa a ser o maior consumidor
Como o modelo interfere em BUILD e PROBE?	O BUILD recebe a relação completa sem custo extra, porque ela já está materializada. A saída do PROBE é materializada inteira: com 10 milhões de funcionários, 1,75 GiB reservados	O BUILD continua igual. O PROBE deixa de produzir uma tabela; o custo por linha passa a incluir as chamadas `next()` (pipeline) ou é amortizado por lote (vetorizado)

Tabela 4: Hash Join e modelo de execução.

3.4 GROUP BY e AGGREGATE

`materialized_group_by` aceita uma ou mais colunas de agrupamento (inteiras ou texto) e uma função (`COUNT`, `SUM`, `MIN`, `MAX` ou `AVG`). A saída é uma tabela com as colunas de grupo seguidas de uma coluna `DOUBLE` com o resultado, e os grupos ficam na ordem da primeira ocorrência. A busca linear pelos grupos da versão anterior ($O(n \times g)$) foi trocada por um índice hash de grupos (`heads`, `next` e estados, que dobram de tamanho quando enchem), com tempo esperado $O(n)$. A mudança foi necessária para o experimento de BUILD: com 100 000 departamentos, a busca linear faria até 600 mil $\times$ 100 mil comparações só no GROUP BY. O índice e os estados são liberados antes de o operador retornar. A consulta isolada `SELECT department_id, AVG(salary) FROM funcionarios GROUP BY department_id` é coberta pelos testes, e a integrada aparece no fim do plano.

4 Decisões de implementação

4.1 Tabela compacta

Na etapa 2, cada linha ocupava 16 valores de 136 bytes (2176 bytes), qualquer que fosse o esquema. Assim, só a tabela base de 10 milhões de funcionários ocuparia cerca de 20 GiB, e a cópia do SCAN mais 20 GiB, numa máquina de 16 GiB. A linha compacta, já listada como otimização no relatório da etapa 2, foi implementada:

```
typedef struct {
    Schema schema;
    Value *values;   /* size * schema.column_count valores, linha a linha */
    size_t size;
    size_t capacity;
} Table;
```

- A linha i começa em `values + i * column_count`, e o acesso é feito por `table_cell`, `table_row_const` e `table_row_mut` (`static inline` em `table.h`).
- `Row` continua existindo, mas só como tupla de montagem na pilha; `table_append` copia apenas as colunas do esquema.
- `VALUE_STRING_MAX_LEN` caiu de 128 para 24, e `sizeof(Value)` de 136 para 32 bytes. O leitor de CSV passou a rejeitar strings com 24 caracteres ou mais, em vez de truncar sem avisar. Os nomes gerados (`department_100000`) têm até 17.

Com isso, `funcionarios` passou de 2176 para 128 bytes por linha, 17 vezes menos.

4.2 Outras decisões

- **Capacidade exata.** SCAN e PROJECT reservam `input->size` linhas de uma vez, e o leitor de CSV ajusta o vetor ao tamanho final (`table_shrink_to_fit`). FILTER, JOIN e GROUP BY não conhecem o tamanho da saída e continuam dobrando a capacidade.
- **BUILD na menor relação**, como pede o enunciado, decidido em tempo de execução pelo número de linhas das entradas já materializadas.
- **Tabela hash em dois vetores**, em vez de um `malloc` por nó: duas alocações por JOIN, liberação simples e melhor localidade.
- **Função de hash única** (`value_hash`, djb2 sobre os bytes do valor seguida de uma etapa de mistura), usada pelo JOIN e pelo GROUP BY. Inteiros e `double` de mesmo valor têm o mesmo hash, coerente com `value_equals`.
- **Ordem do plano.** JOIN $\rightarrow$ PROJECT $\rightarrow$ GROUP BY, como no pipeline conceitual do enunciado. A versão anterior do `main.c` agrupava antes de projetar.
- **Liberação antecipada no benchmark.** Cada intermediário é liberado assim que o consumidor termina. O pico de memória passa a ser o maior conjunto de tabelas vivas ao mesmo tempo, e não a soma de todas.

5 Gerenciamento de memória

5.1 Quem aloca e quem libera

A Tabela 5 resume a posse de cada recurso.

Recurso	Quem aloca	Dono	Quem libera
Vetor de valores das tabelas base	`table_load_csv`	chamador	`table_free`, no fim
Saída de SCAN, FILTER, PROJECT, JOIN e GROUP BY	o operador	o chamador (`output`)	o chamador, depois do último consumidor
Tabela hash (`heads`, `entries`)	`hash_build`	o Hash Join	`hash_table_free`, antes de retornar (também em erro)
Índice de grupos	`materialized_group_by`	o GROUP BY	`group_index_free`, antes de retornar (também em erro)
`Row` de montagem	pilha	a função	fim do escopo
Strings	dentro do `Value`	a tabela	junto com o vetor de valores

Tabela 5: Alocação e liberação de memória.

No benchmark, a ordem de liberação é: `scan_f` depois do FILTER; `filtered` e `scan_d` depois do JOIN; `joined` depois do PROJECT; `projected` depois do GROUP BY. Não há buffer reutilizado entre operadores. Ponteiros obtidos por `table_cell` e similares valem até o próximo crescimento da tabela ou até `table_free`, e nenhum operador guarda ponteiro de entrada depois de retornar. `input == output` continua sendo rejeitado.

A métrica `memory_bytes` mede a capacidade reservada (capacidade × bytes por linha). O pico de RSS do processo (`VmHWM`, que inclui as tabelas base) pode ficar abaixo da soma reservada, porque páginas reservadas e nunca escritas não ocupam memória física. Por isso os experimentos reportam as duas medidas.

5.2 Validação com Valgrind

O Valgrind Memcheck [3] foi executado no código final desta etapa (Tabela 6); os logs completos estão em `results/validacao/`. `make sanitize` (AddressSanitizer e UndefinedBehaviorSanitizer) terminou com `all materialized tests passed`. A diferença de 2 alocações entre os dois JOINs no benchmark corresponde aos dois vetores da tabela hash.

Programa	Alocações	Liberações	Bytes alocados	Em uso	Erros
`test_materialized`	218	218	12 881 115	0	0
`miniquery` (dataset versionado)	26	26	38 168	0	0
`miniquery_bench`, HJ, 10 mil linhas	74	74	13 548 048	0	0
`miniquery_bench`, NLJ, 10 mil linhas	72	72	13 516 040	0	0

Tabela 6: Resultados do Valgrind Memcheck.

6 Instrumentação

A estrutura `JoinMetrics` registra o que a seção 5 do enunciado pede, e alguns campos extras (Tabela 7). `batch_size` e `batches_processed` são do modelo vetorizado e não se aplicam aqui. `QueryMetrics` (SCAN, FILTER, PROJECT e GROUP BY) registra `rows_in`, `rows_out`, `memory_bytes` e `time_ms`. O benchmark mede ainda o tempo de parede da consulta inteira com `timespec_get` e o pico de RSS do processo.

Campo	Significado
`build_rows`, `probe_rows`	Linhas de BUILD e PROBE (no Nested Loop, relação interna e externa)
`rows_out`	Linhas produzidas pelo JOIN
`build_time_ms`, `probe_time_ms`, `total_time_ms`	Tempo de CPU (`clock()`) de cada fase e do operador inteiro
`memory_bytes`	Tabela hash mais a saída reservada
`hash_table_entries`, `hash_table_bytes`	Entradas e bytes da tabela hash
`hash_buckets`, `key_comparisons`, `output_bytes`, `build_is_left`	Extras: buckets, comparações de chave executadas, bytes da saída e lado escolhido para BUILD

Tabela 7: Métricas do JOIN.

A saída do `./miniquery` sobre o dataset versionado (8 funcionários e 5 departamentos) segue o formato do enunciado; o bloco `[NESTED_LOOP_JOIN]` sai antes, com `key_comparisons=25` (5×5). Os tempos variam entre execuções.

```
SCAN f: 8 linhas | SCAN d: 5 linhas | FILTER salary > 5000.00: 5 linhas

[HASH_JOIN]
build_rows=5
probe_rows=5
hash_table_entries=5
hash_table_bytes=168
build_time_ms=0.001
probe_time_ms=0.002
total_time_ms=0.004
rows_out=6
memory_bytes=3752
key_comparisons=6
```

7 Testes automatizados

`make test` compila a suíte com `-std=c11 -Wall -Wextra -Wpedantic -Werror -O2`, roda 14 grupos de teste e termina com `all materialized tests passed`:

- **Etapa 2:** CSV e esquema, cópia independente do SCAN, FILTER, PROJECT, agregações e pipeline completo.
- **JOIN:** aliases e esquema resultante, chaves repetidas, chave sem par, os dois lados possíveis de BUILD, entrada vazia e equivalência por multiconjunto entre Nested Loop e Hash Join em $2\,000 \times 300$ linhas pseudoaleatórias, com chaves repetidas e chaves sem par.
- **Métricas:** `key_comparisons` $= |R| \times |S|$ no Nested Loop, tamanho exato da tabela hash e `memory_bytes` igual a hash mais saída.
- **GROUP BY:** as cinco funções por grupo, chave inteira e texto, e 20 000 linhas em 5 000 grupos, para forçar o crescimento do índice e conferir a ordem e as médias.
- **Armazenamento compacto:** bytes por linha, capacidade exata e rejeição de string longa demais no CSV.
- **Consulta completa** na ordem do enunciado, com os valores esperados (`engineering 6500`, `finance 7600`, `finance-remote 7600`, `operations 9100`).

Além disso, os experimentos conferem a cada configuração que os dois algoritmos devolvem o mesmo número de linhas e a mesma soma das médias por grupo. Não houve divergência.

8 Experimentos

8.1 Configuração utilizada

Item	Valor
CPU	Intel Xeon (KVM), 8 CPUs lógicas; L1d 48 KiB e L2 2 MiB por núcleo; L3 320 MiB
Memória	15,6 GiB (12,7 GiB disponíveis no início)
Sistema	Debian GNU/Linux 13 (trixie), kernel 6.12, máquina virtual compartilhada
Compilador	GCC 14.2.0, `-std=c11 -Wall -Wextra -Wpedantic -Werror -O2`
Execução	1 thread, sequencial, um processo por configuração (dataset, algoritmo, limite)
Repetições	1 de aquecimento (descartada) e 5 medidas, com mediana; sem aquecimento no Nested Loop acima de 5×10^9 comparações
Data	11/11/2026; detalhes em `results/ambiente.txt`

Tabela 8: Ambiente dos experimentos.

Os datasets base são carregados uma vez por processo, fora do tempo medido, e cada repetição executa a consulta inteira de novo, com alocações novas. A máquina é virtual e compartilhada, então há ruído: a mesma configuração (1 milhão de funcionários, 1 000 departamentos, `salary > 5000`) medida em dois processos diferentes deu 254,5 e 250,8 ms com Hash Join e 1 821,7 e 1 998,0 ms com Nested Loop, cerca de 10% de diferença.

8.2 Datasets

`bench/generate_data.py` gera dados determinísticos (semente 2026). Em `departamentos`, os ids vão de 1 a D, com `name = department_<id>` e `region = region_<id % 5>`. Em `funcionarios`, `department_id` é uniforme em $1..D$ (toda linha tem par), `salary` é uniforme em $[1000, 11000)$ e `age` em 18..70. Assim, `salary > t` mantém cerca de $(11000 - t)/10000$ das linhas.

Arquivo	Linhas	Tamanho
`funcionarios_10k_d1k.csv`	10 000	0,2 MB
`funcionarios_100k_d1k.csv`	100 000	2,1 MB
`funcionarios_1m_d1k.csv` (e `_d100`, `_d10k`, `_d100k`)	1 000 000 cada	21 a 24 MB
`funcionarios_5m_d1k.csv`	5 000 000	114 MB
`funcionarios_10m_d1k.csv`	10 000 000	229 MB
`departamentos_100.csv` a `departamentos_100k.csv`	100 a 100 000	2,6 KB a 3,2 MB

Tabela 9: Datasets dos experimentos.

`results/datasets.csv` registra o tamanho e o SHA-256 de cada arquivo, e `make experiments` recria exatamente os mesmos bytes. Os arquivos de 5 e 10 milhões passam dos limites de um repositório comum (o GitHub recusa arquivos acima de 100 MB). Por isso ficam versionados só os datasets pequenos, em `data/experimentos/`, além do gerador com a semente. Compactado, o arquivo de 10 milhões tem 93 MB com gzip e 69 MB com xz.

8.3 Consulta, algoritmos e métricas

Todas as medições usam a consulta completa da seção 7 do enunciado, mudando só o limite do FILTER. O plano é `SCAN f` → `FILTER` → `JOIN` ← `SCAN d` → `PROJECT (d.name, f.salary)` → `GROUP BY/AVG`, executado com Nested Loop Join (externa: `funcionarios` filtrado; interna: `departamentos`) e com Hash Join (BUILD: a menor entrada, que foi sempre `departamentos`; PROBE: `funcionarios` filtrado).

Por repetição são registrados (`results/raw/*.csv`): linhas de cada operador, tempo de CPU de cada operador (SCAN f, SCAN d, FILTER, BUILD, PROBE, JOIN, PROJECT e GROUP BY), tempo de parede da consulta, comparações de chave, entradas e bytes da tabela hash, bytes reservados por intermediário, pico de intermediários vivos, pico de RSS e uma soma de verificação das médias. Nas tabelas a seguir, "Consulta" é a mediana do tempo de parede da consulta inteira, "Pico interm."

é o maior conjunto de tabelas intermediárias vivas ao mesmo tempo e “Pico RSS” é a memória física máxima do processo, incluindo as tabelas base. Todos os tempos estão em milissegundos.

8.4 Cardinalidade

Com 1 000 departamentos e `salary > 5000` (cerca de 60% das linhas), a quantidade de funcionários variou de 10 mil a 10 milhões. Todas as cardinalidades pedidas foram executadas, inclusive 10 milhões com Nested Loop (6×10^9 comparações, cerca de 20 s por repetição). As Tabelas 10 a 12 e as Figuras 2 a 4 trazem os resultados (HJ: Hash Join; NLJ: Nested Loop Join).

Funcionários	Alg.	Linhas PROBE	Comparações	BUILD	PROBE	JOIN	Consulta	min–max
10 000	HJ	5 862	8 565	0,010	0,7	0,7	**2,0**	1,9–2,0
10 000	NLJ	5 862	5 862 000	0,000	17,5	17,5	**19,0**	18,4–23,6
100 000	HJ	60 051	88 121	0,016	9,6	9,6	**29,3**	27,8–34,0
100 000	NLJ	60 051	60 051 000	0,000	168,4	168,4	**185,0**	181,2–195,6
1 000 000	HJ	600 178	879 899	0,027	71,6	71,6	**254,5**	246,0–315,4
1 000 000	NLJ	600 178	600 178 000	0,000	1 645,6	1 645,6	**1 821,7**	1 786,6–1 971,6
5 000 000	HJ	3 000 405	4 396 861	0,029	367,6	367,6	**1 290,3**	1 221,3–1 415,8
5 000 000	NLJ	3 000 405	3 000 405 000	0,000	8 482,0	8 482,0	**9 399,0**	9 303,7–9 632,0
10 000 000	HJ	5 999 764	8 794 780	0,027	741,2	741,2	**2 711,9**	2 656,2–2 869,3
10 000 000	NLJ	5 999 764	5 999 764 000	0,000	17 773,9	17 773,9	**19 929,8**	19 323,4–20 357,6

Tabela 10: Experimento de cardinalidade: linhas, comparações e tempos (ms, mediana de 5 repetições).

Funcionários	Alg.	SCAN f	SCAN d	FILTER	JOIN	PROJECT	GROUP BY	Soma	JOIN/soma
10 000	HJ	0,3	0,0	0,4	0,7	0,1	0,4	1,9	36%
10 000	NLJ	0,4	0,0	0,4	17,5	0,2	0,4	18,9	93%
100 000	HJ	6,4	0,0	5,4	9,6	2,7	3,6	28,1	34%
100 000	NLJ	5,7	0,0	5,2	168,4	2,5	2,7	184,1	91%
1 000 000	HJ	57,5	0,0	48,9	71,6	37,1	25,4	245,4	29%
1 000 000	NLJ	55,6	0,0	49,7	1 645,6	36,3	25,4	1 812,8	91%
5 000 000	HJ	282,9	0,0	289,2	367,6	181,8	127,8	1 247,5	29%
5 000 000	NLJ	287,5	0,0	249,6	8 482,0	187,4	126,2	9 337,2	91%
10 000 000	HJ	580,8	0,0	634,7	741,2	362,7	257,8	2 610,9	28%
10 000 000	NLJ	727,1	0,0	565,7	17 773,9	432,2	250,2	19 808,5	90%

Tabela 11: Experimento de cardinalidade: tempo de CPU por operador (ms, mediana).

Funcionários	SCAN f+d	FILTER	JOIN (saída)	Hash (KiB)	PROJECT	Total	Pico interm.	Pico RSS
10 000	1,3	1,0	1,8	31,3	0,4	4,6	2,9	6,6
100 000	12,3	8,0	14,0	31,3	3,7	38,1	22,1	50,0
1 000 000	122,2	128,0	224,0	31,3	36,6	510,9	352,1	325,5
5 000 000	610,4	512,0	896,0	31,3	183,1	2 201,7	1 408,1	1 619,6
10 000 000	1 220,8	1 024,0	1 792,0	31,3	366,2	4 403,1	2 816,1	3 236,7

Tabela 12: Experimento de cardinalidade: memória com Hash Join (MiB, salvo indicação). “Total” é a soma reservada por todos os intermediários.

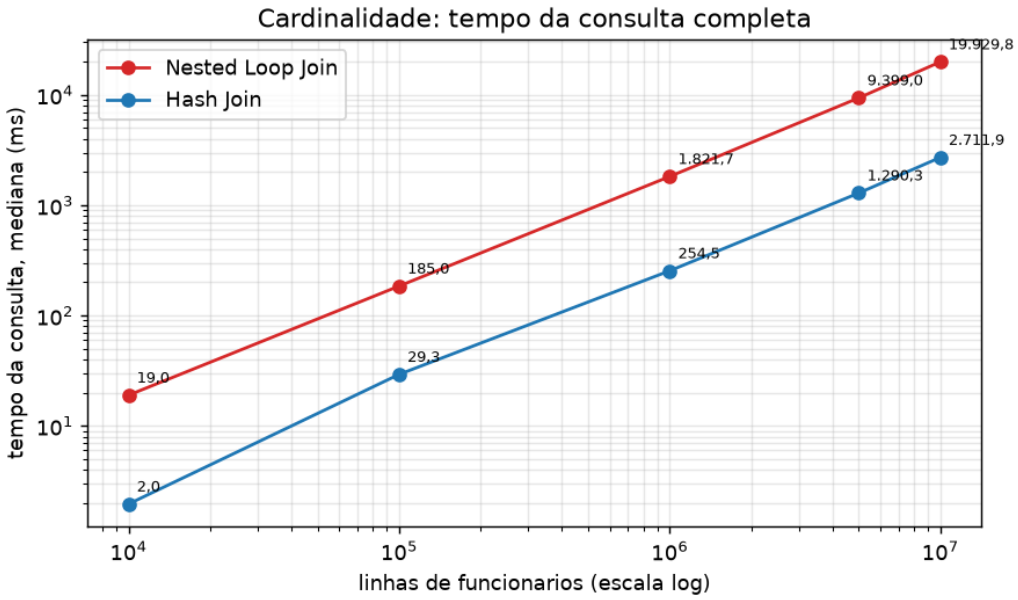


Figura 2: Cardinalidade: tempo da consulta (mediana).

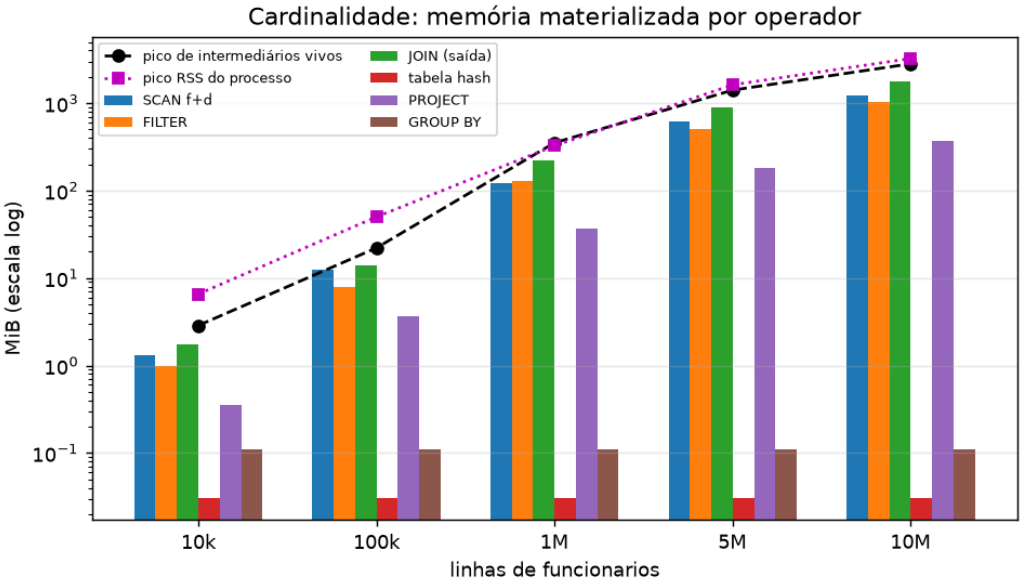


Figura 3: Cardinalidade: memória reservada pelos intermediários e pico de RSS.

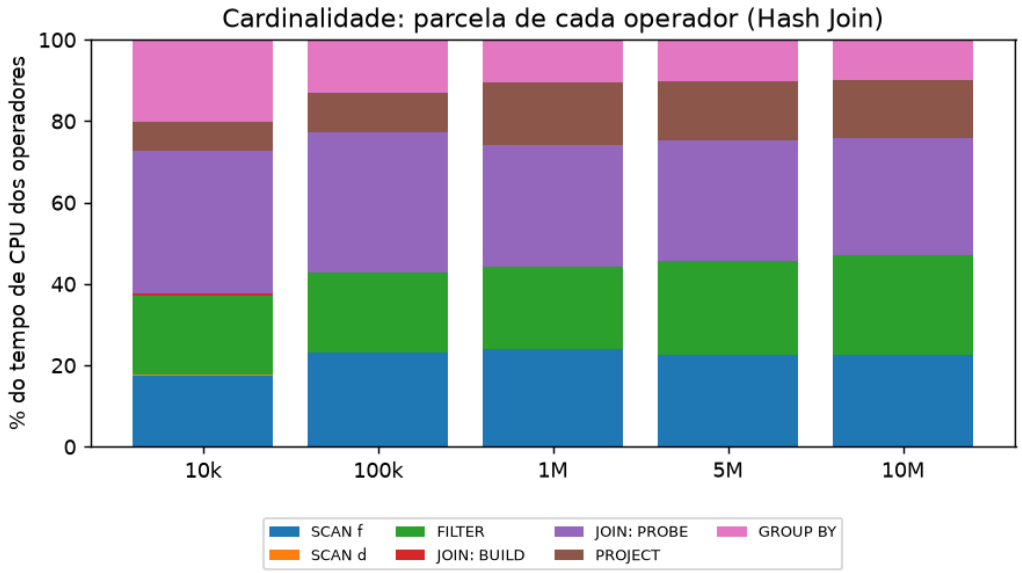


Figura 4: Cardinalidade: parcela de cada operador no tempo.

O tempo cresce de forma linear com o número de funcionários nos dois algoritmos, porque o número de departamentos está fixo. O Nested Loop faz $|F'| \times 1\,000$ comparações (cerca de 3 ns cada) e gasta de 90 a 93% do tempo dos operadores no JOIN. O Hash Join faz cerca de 1,47 comparação por linha de PROBE e gasta de 28 a 36% no JOIN. A diferença entre os dois fica estável em cerca de 7 vezes a partir de 100 mil linhas. A memória também cresce de forma linear: com 10 milhões, a consulta reserva 4,3 GiB de intermediários para devolver 1 000 grupos, e o pico de intermediários vivos chega a 2,75 GiB durante o JOIN, quando a saída do FILTER e a do JOIN coexistem.

8.5 Seletividade

Com 1 milhão de funcionários e 1 000 departamentos, os limites 10 900, 10 000, 6 000 e 2 000 mantêm 1, 10, 50 e 90% das linhas. As seletividades medidas foram 0,98%, 9,95%, 50,0% e 90,0%.

Seletividade	Alg.	Linhas PROBE	Comparações	BUILD	PROBE	JOIN	Consulta	min–max
1,0%	HJ	9 798	14 244	0,023	1,1	1,2	**80,6**	78,5–82,1
1,0%	NLJ	9 798	9 798 000	0,000	35,0	35,0	**128,9**	103,6–150,4
10,0%	HJ	99 519	145 589	0,024	12,1	12,2	**104,6**	103,0–124,8
10,0%	NLJ	99 519	99 519 000	0,000	275,7	275,7	**375,8**	370,9–455,9
50,0%	HJ	500 004	732 986	0,027	66,3	66,3	**220,4**	217,7–233,3
50,0%	NLJ	500 004	500 004 000	0,000	1 408,0	1 408,0	**1 569,4**	1 520,7–1 623,5
90,0%	HJ	900 002	1 319 626	0,026	106,5	106,6	**324,9**	321,6–328,5
90,0%	NLJ	900 002	900 002 000	0,000	2 530,0	2 530,0	**2 781,2**	2 729,0–3 079,5

Tabela 13: Experimento de seletividade: linhas, comparações e tempos (ms, mediana de 5 repetições).

Seletividade	Alg.	SCAN f	SCAN d	FILTER	JOIN	PROJECT	GROUP BY	Soma	JOIN/soma
1,0%	HJ	56,9	0,0	17,9	1,2	0,2	0,5	77,0	2%
1,0%	NLJ	63,8	0,0	20,5	35,0	0,4	0,6	124,5	28%
10,0%	HJ	56,2	0,0	23,9	12,2	3,8	5,1	101,4	12%
10,0%	NLJ	59,9	0,0	25,0	275,7	4,2	4,5	372,5	74%
50,0%	HJ	56,1	0,0	49,0	66,3	21,6	21,8	214,3	31%
50,0%	NLJ	57,3	0,0	54,6	1 408,0	20,6	21,4	1 561,4	90%
90,0%	HJ	55,9	0,0	60,1	106,6	53,8	37,6	313,6	34%
90,0%	NLJ	61,0	0,0	64,8	2 530,0	61,4	39,5	2 763,1	92%

Tabela 14: Experimento de seletividade: tempo de CPU por operador (ms, mediana).

Seletividade	SCAN f+d	FILTER	JOIN (saída)	Hash (KiB)	PROJECT	Total	Pico interm.	Pico RSS
1,0%	122,2	2,0	3,5	31,3	0,6	128,4	124,2	247,6
10,0%	122,2	16,0	28,0	31,3	6,1	172,4	138,2	264,1
50,0%	122,2	64,0	112,0	31,3	30,5	328,8	186,2	339,0
90,0%	122,2	128,0	224,0	31,3	54,9	529,2	352,1	426,2

Tabela 15: Experimento de seletividade: memória com Hash Join (MiB, salvo indicação). “Total” é a soma reservada por todos os intermediários.

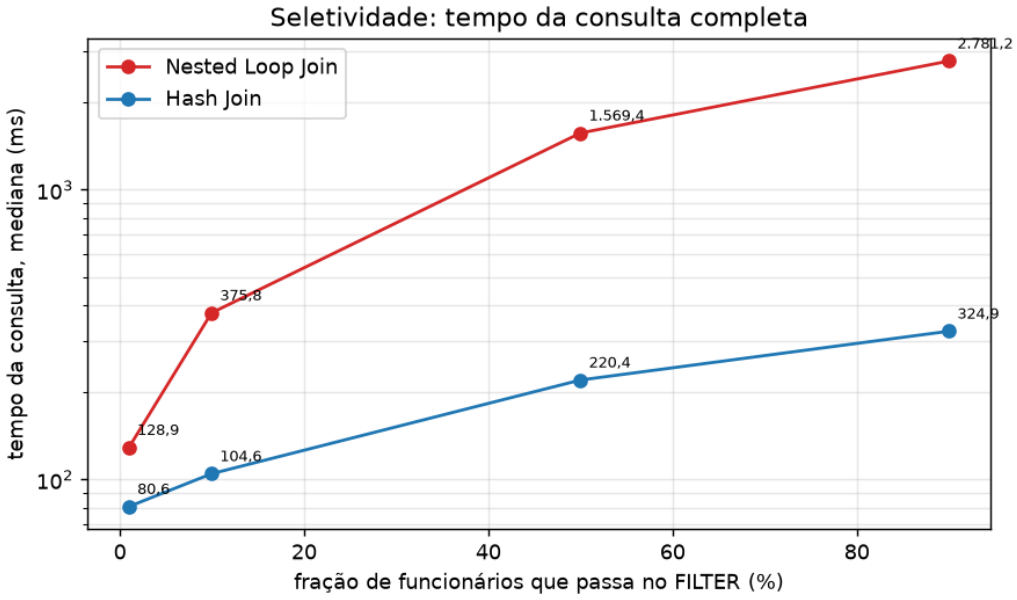


Figura 5: Seletividade: tempo da consulta (mediana).

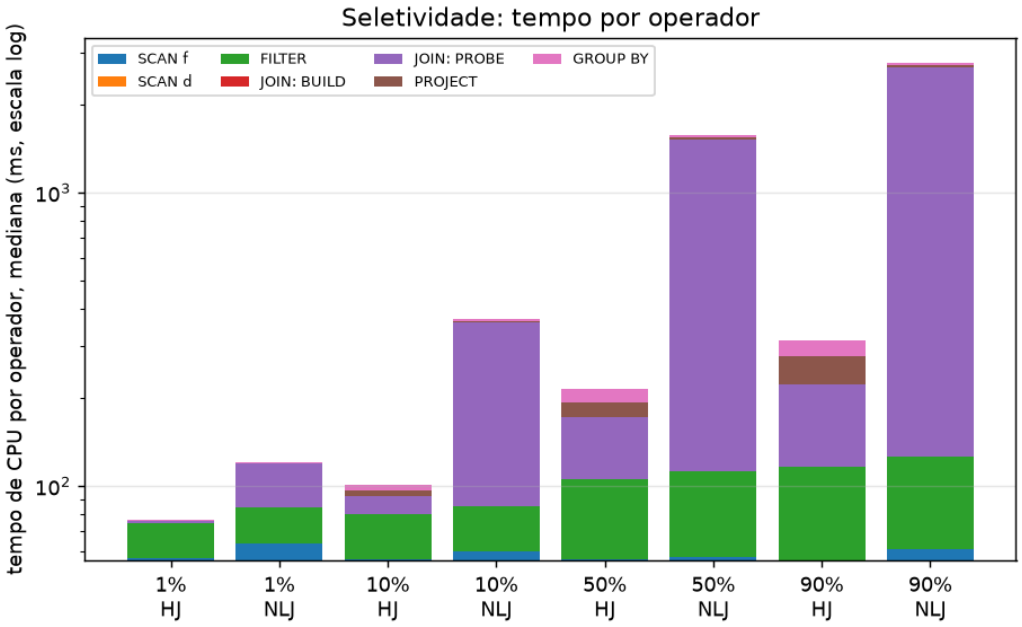


Figura 6: Seletividade: tempo por operador.

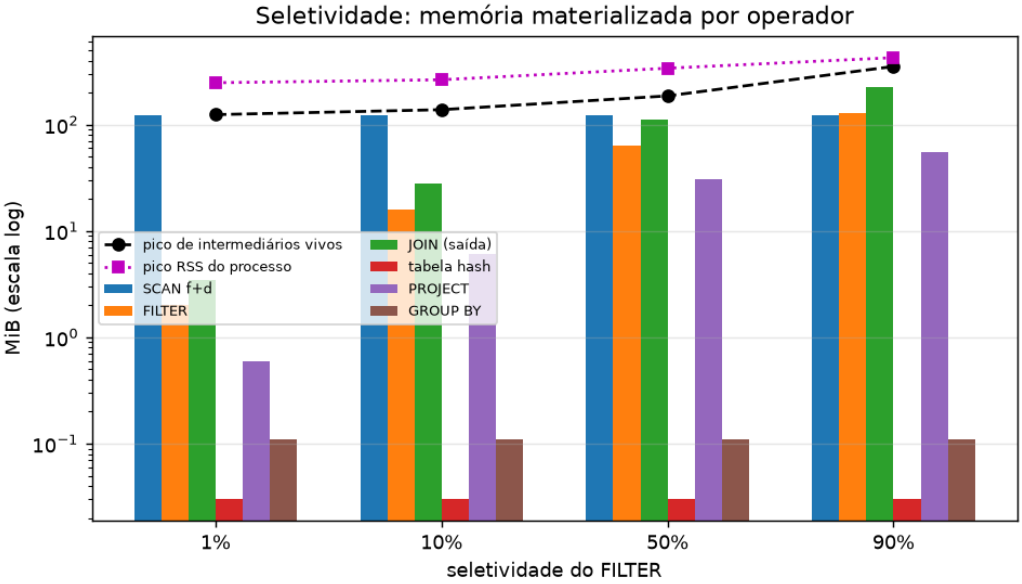


Figura 7: Seletividade: memória reservada pelos intermediários.

O FILTER decide quantas linhas chegam ao JOIN. De 1% para 90%, a entrada do JOIN cresce 92 vezes, e o tempo do JOIN acompanha: de 35 para 2 530 ms no Nested Loop e de 1,2 para 106,6 ms no Hash Join. Já o tempo da consulta com Hash Join cresce só 4 vezes (80,6 para 324,9 ms), porque o SCAN de `funcionarios` custa sempre cerca de 56 ms e reserva sempre 122 MiB, qualquer que seja o filtro. Com 1%, SCAN e FILTER somam 97% do tempo dos operadores. É o custo fixo da materialização: a cópia completa feita antes de o filtro descartar qualquer linha. A vantagem do Hash Join sobre o Nested Loop cresce com a seletividade: 1,6 vez com 1%, 3,6 com 10%, 7,1 com 50% e 8,6 com 90%.

8.6 Tamanho da relação de BUILD

Com 1 milhão de funcionários e `salary > 5000`, o número de departamentos variou de 100 a 100 000. Como todo funcionário tem departamento, a saída do JOIN fica perto de 600 mil linhas em todos

os casos, e só o tamanho do BUILD muda. O Nested Loop com 100 000 departamentos (6×10^{10} comparações) foi executado por completo: 5 repetições, sem aquecimento, cerca de 4 minutos cada.

Departamentos	Alg.	Linhas PROBE	Comparações	BUILD	PROBE	JOIN	Consulta	min–max
100	HJ	599 745	852 072	0,004	70,1	70,1	**252,0**	246,0–275,9
100	NLJ	599 745	59 974 500	0,000	214,2	214,2	**388,5**	380,9–404,3
1 000	HJ	600 178	879 899	0,026	71,2	71,2	**250,8**	246,9–282,8
1 000	NLJ	600 178	600 178 000	0,000	1 809,1	1 809,1	**1 998,0**	1 968,1–2 088,9
10 000	HJ	599 676	1 154 395	0,216	112,1	112,3	**315,2**	293,3–420,0
10 000	NLJ	599 676	5 996 760 000	0,000	16 565,4	16 565,4	**16 825,0**	16 050,3–18 433,4
100 000	HJ	599 556	1 273 291	2,896	135,5	138,4	**387,2**	378,6–388,3
100 000	NLJ	599 556	59 955 600 000	0,000	248 502,4	248 502,4	**249 065,3**	235 688,7–257 660,8

Tabela 16: Experimento de tamanho do BUILD: linhas, comparações e tempos (ms, mediana de 5 repetições).

Departamentos	Alg.	SCAN f	SCAN d	FILTER	JOIN	PROJECT	GROUP BY	Soma	JOIN/soma
100	HJ	61,5	0,0	51,5	70,1	37,4	20,7	242,9	29%
100	NLJ	56,8	0,0	49,8	214,2	37,2	20,4	380,3	56%
1 000	HJ	58,7	0,0	50,6	71,2	36,9	25,3	240,7	30%
1 000	NLJ	63,7	0,0	56,9	1 809,1	40,5	26,6	1 985,1	91%
10 000	HJ	61,0	0,5	56,0	112,3	38,9	31,0	296,9	38%
10 000	NLJ	73,5	0,6	61,9	16 565,4	49,7	31,2	16 809,1	99%
100 000	HJ	60,3	4,6	53,1	138,4	36,0	83,3	376,1	37%
100 000	NLJ	134,1	9,4	78,7	248 502,4	114,4	88,1	248 981,5	100%

Tabela 17: Experimento de tamanho do BUILD: tempo de CPU por operador (ms, mediana).

Departamentos	SCAN f+d	FILTER	JOIN (saída)	Hash (KiB)	PROJECT	Total	Pico interm.	Pico RSS
100	122,1	128,0	224,0	3,1	36,6	510,7	352,0	325,2
1 000	122,2	128,0	224,0	31,3	36,6	510,9	352,1	325,5
10 000	123,0	128,0	224,0	312,5	36,6	513,6	353,2	328,1
100 000	131,2	128,0	224,0	3 125,0	36,6	536,9	364,2	353,4

Tabela 18: Experimento de tamanho do BUILD: memória com Hash Join (MiB, salvo indicação). "Total" é a soma reservada por todos os intermediários.

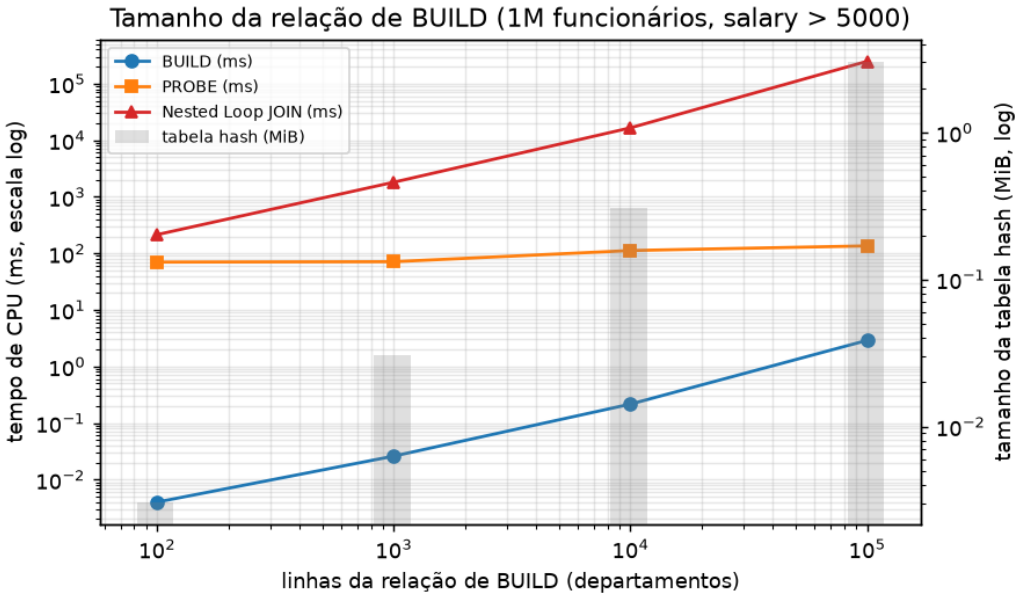


Figura 8: Tamanho do BUILD: tempo de BUILD e PROBE e tamanho da tabela hash.

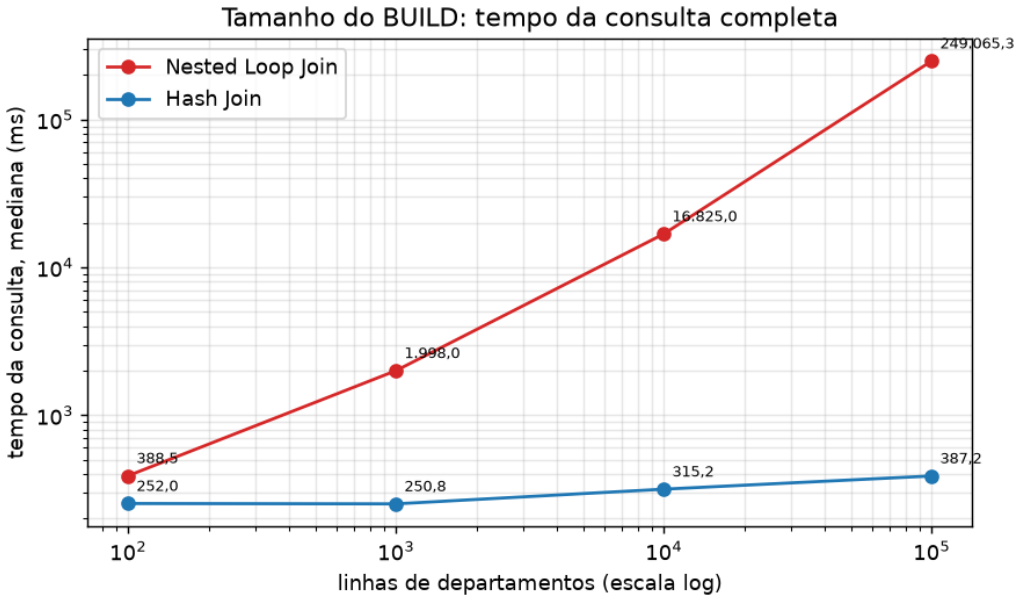


Figura 9: Tamanho do BUILD: tempo da consulta (mediana).

O BUILD é barato e cresce de forma linear: 0,004 ms com 100 linhas e 2,9 ms com 100 mil, com a tabela hash indo de 3,1 KiB para 3,1 MiB. O PROBE quase dobra (de 70 para 135 ms). Com 100 mil chaves, a tabela hash (3,1 MiB) e as 100 mil linhas de `departamentos` (9,2 MiB) já não cabem no cache L2 de 2 MiB, e cada linha de PROBE faz mais acessos aleatórios. O número de comparações por linha de PROBE também sobe de 1,42 para 2,12, um pouco acima do 1,5 esperado para carga 0,5, o que indica que a função de hash espalha pior as chaves maiores. No Nested Loop, o JOIN cresce na proporção de $|D|$: de 214 ms com 100 para 248 502 ms (4,1 minutos) com 100 mil departamentos, e o custo por comparação sobe de cerca de 3 para 4,1 ns quando a relação interna deixa de caber no cache. Com 100 departamentos, o Nested Loop deixa a consulta só 1,5 vez mais lenta que o Hash Join; com 100 mil, 643 vezes.

8.7 Tamanho do batch e comparações obrigatórias

O teste de tamanho de batch (seção 8.4 do enunciado) não se aplica ao modelo materializado, porque não há lotes entre operadores, e o enunciado o atribui ao responsável pelo modelo vetorizado. A Tabela 19 resume a situação das comparações da seção 9 do enunciado.

Comparação	Situação
Nested Loop × Hash Join	Medida (cardinalidade, seletividade e BUILD)
Materialização × Pipeline × Vetorizado	Não medida aqui: depende das implementações de JOIN dos Grupos B e C
Cardinalidades, seletividades e tamanhos de BUILD	Medidas
Tamanhos de batch	Não se aplica ao Grupo A
Memória, tempo total, tempo por operador, dados intermediários	Medidos (tabelas de tempo, de operadores e de memória)

Tabela 19: Comparações exigidas pelo enunciado.

Para que a comparação entre modelos seja justa, os três grupos precisam usar os mesmos datasets e a mesma consulta. O gerador e a semente desta etapa ficam no repositório para isso.

9 Resposta à questão da etapa

Como o modelo de execução e o algoritmo de JOIN influenciam o custo de uma consulta composta por scan, filtro, join, projeção e agregação?

Nos experimentos do modelo materializado, o algoritmo de JOIN decide a ordem de grandeza do custo, e o modelo de execução decide o custo que sobra e, sobretudo, a memória.

1. O algoritmo de JOIN define a complexidade. O Nested Loop compara cada linha externa com todas as internas, então o custo é proporcional a $|F'| \times |D|$. O Hash Join paga $|D|$ para construir a tabela e cerca de 1,5 comparação por linha de PROBE. Com 1 milhão de funcionários e 1 000 departamentos, foram 600 milhões de comparações contra 880 mil (682 vezes menos), e o JOIN caiu de 1 645,6 para 71,6 ms (23 vezes). O ganho em tempo é menor que o ganho em comparações porque cada passo do Hash Join custa mais (cálculo do hash, acesso aleatório à cadeia) e porque os dois algoritmos pagam a mesma materialização da saída (224 MiB reservados). O tamanho da relação interna é o fator decisivo: com 100 departamentos, o Nested Loop deixa a consulta só 1,5 vez mais lenta; com 100 mil, 643 vezes (249 s contra 0,39 s). O Hash Join quase não sente essa variação (de 252 para 387 ms), porque o BUILD custa no máximo 2,9 ms.

2. Filtrar antes do JOIN reduz o custo do JOIN na mesma proporção. De 90% para 1% de seletividade, o JOIN do Nested Loop cai de 2 530 para 35 ms (72 vezes) e o do Hash Join de 106,6 para 1,2 ms. Quanto mais caro o JOIN por linha, mais o filtro compensa; por isso a vantagem do Hash Join cai de 8,6 vezes (90%) para 1,6 vez (1%).

3. Com Hash Join, o que sobra é o custo do modelo materializado. Quando o JOIN é eficiente, ele responde por só 28 a 38% do tempo. O resto é SCAN, FILTER, PROJECT e GROUP BY, e todos gravam a saída inteira. Só o SCAN de `funcionarios` (cópia completa) e o PROJECT (outra cópia) somam 36% do tempo com 10 milhões de linhas (943 de 2 611 ms). Com seletividade de 1%, SCAN e FILTER são 97% do tempo, porque a tabela inteira é copiada antes de o filtro descartar 99% dela. Na memória, a consulta de 10 milhões reserva 4,3 GiB de intermediários (SCAN 1,19 GiB, FILTER 1 GiB, JOIN 1,75 GiB, PROJECT 366 MiB) para devolver 1 000 linhas. O pico de intermediários vivos chega a 2,75 GiB, e o pico de RSS do processo a 3,2 GiB. A tabela hash ocupa só 31 KiB desse total: no modelo materializado, quem consome memória são as tabelas intermediárias, e não a estrutura do JOIN.

4. O modelo muda o papel de BUILD e PROBE. No modelo materializado, o BUILD não espera mais do que esperaria de qualquer forma, porque a relação de `departamentos` já está completa. O PROBE, porém, também recebe a entrada inteira e materializa toda a saída, que é o maior intermediário da consulta. Nos modelos pipeline e vetorizado, que não foram medidos aqui, o

esperado pelo funcionamento dos operadores é que o BUILD continue sendo um ponto de bloqueio (a tabela hash precisa estar completa antes do primeiro PROBE), mas que o PROBE, o PROJECT e o GROUP BY passem a consumir as linhas à medida que elas chegam. Assim, os gigabytes de tabelas intermediárias dariam lugar à tabela hash e ao estado dos grupos (cerca de 145 KiB com 1 000 departamentos), mais um lote por operador no vetorizado. Em troca, haveria custo de chamada por tupla (pipeline) ou por lote (vetorizado). Quanto isso custa de fato só pode ser respondido com as medições dos Grupos B e C nos mesmos datasets.

Conclusão. O custo da consulta é aproximadamente o custo do JOIN mais um custo proporcional ao volume de dados copiado entre os operadores. O algoritmo de JOIN define o primeiro termo: o Nested Loop o torna quadrático e dominante (90 a 100% do tempo quando $|D| \geq 1\,000$), e o Hash Join o torna linear e secundário. O modelo de execução define o segundo: no modelo materializado, cada operador copia a sua saída inteira, o que, com Hash Join, responde pela maior parte do tempo e por toda a memória que cresce com os dados. Filtros seletivos antes do JOIN reduzem os dois termos, exceto a cópia do SCAN, que a materialização paga sempre.

10 Dificuldades encontradas

- **Memória da linha fixa.** A `Row` de 2176 bytes da etapa 2 inviabilizava as cardinalidades pedidas: 10 milhões de linhas ocupariam cerca de 20 GiB por tabela. A solução foi a tabela compacta (Seção 4.1), que exigiu trocar todos os acessos `rows[i].values[j]` por funções de acesso.
- **GROUP BY linear.** A busca linear pelos grupos funcionava nos testes, mas tornaria o experimento com 100 mil departamentos inviável. Foi trocada por um índice hash, mantendo a ordem dos grupos que os testes já verificavam.
- **Custo do Nested Loop nos experimentos.** Com 100 mil departamentos, cada execução leva cerca de 4 minutos. Mesmo assim, as 5 repetições foram feitas, só sem a rodada de aquecimento.
- **Medição.** `clock()` mede tempo de CPU por operador, e o tempo da consulta inteira precisou de relógio de parede (`timespec_get`). A memória reservada e a física são diferentes, por isso ambas são reportadas. A máquina compartilhada introduz ruído de até cerca de 10% entre processos.
- **Datasets grandes no GitHub.** Os arquivos de 5 e 10 milhões passam dos limites de um repositório comum, por isso ficam fora do Git e são gerados de forma determinística.

11 Limitações atuais

- Só o modelo materializado, sem comparação medida com pipeline e vetorizado.
- Só INNER JOIN com uma condição de igualdade. Não há planner: o algoritmo é escolhido pelo chamador, e só o lado de BUILD é automático.
- Não há parser SQL; as consultas são montadas no código.
- CSV simples: sem aspas, sem `NULL` e sem validação do cabeçalho; strings de até 23 caracteres; no máximo 16 colunas por tabela.
- Não há verificação de memória disponível antes de materializar: uma entrada grande demais termina em falha de alocação (tratada, com liberação).
- Tempos de operador em tempo de CPU; consulta inteira em tempo de parede.
- O FILTER aceita um único predicado.

12 Possíveis otimizações futuras

As otimizações a seguir partem de medições desta etapa.

1. **SCAN sem cópia, ou fundido com o FILTER.** O SCAN custa de 22 a 23% do tempo e 1,19 GiB com 10 milhões; com seletividade de 1%, SCAN e FILTER são 97% do tempo.
2. **Projeção antecipada.** A saída do JOIN tem 7 colunas (224 bytes por linha) e é o maior intermediário, mas só 2 colunas são usadas depois. O JOIN poderia gravar só `d.name` e `f.salary`.
3. **Agregação no próprio PROBE.** Fundir o GROUP BY ao JOIN eliminaria a saída do JOIN e a do PROJECT (2,1 GiB com 10 milhões).
4. **Melhor função de hash ou endereçamento aberto.** Com 100 mil chaves foram 2,12 comparações por linha de PROBE, contra 1,5 esperado.

5. **Estimar a capacidade da saída do FILTER e do JOIN.** A capacidade dobra e chega a reservar quase o dobro do usado: o FILTER de 10 milhões reservou 1 GiB para 732 MiB de linhas.
6. **Strings fora da linha.** O `Value` de 32 bytes existe por causa da string de 24 bytes, e as colunas numéricas pagam esse tamanho.
7. **Armazenamento colunar.** PROJECT e GROUP BY leem 2 de 7 colunas.
8. **Planner simples.** Escolher Nested Loop só quando $|R| \times |S|$ for pequeno.
9. **Hash Join particionado (grace).** Para quando a relação de BUILD não couber na memória.

13 Material de extensão

O material de extensão desta etapa é um vídeo curto, de 3 a 5 minutos, sobre o tema "Nested Loop Join versus Hash Join e a fase de BUILD". O vídeo mostra, no código, os dois laços do Nested Loop e as fases de BUILD e PROBE do Hash Join, e faz duas demonstrações no terminal. A primeira executa a consulta completa com 1 milhão de funcionários e 1 000 departamentos e compara as métricas dos dois JOINs:

```
python3 bench/generate_data.py --employees 1000000 --departments 1000 \
    --employee-file /tmp/demo/funcionarios_1m.csv \
    --department-file /tmp/demo/departamentos_1k.csv
./miniquery /tmp/demo/funcionarios_1m.csv /tmp/demo/departamentos_1k.csv
```

Nessa execução, o Nested Loop faz 600 178 000 comparações e o Hash Join 879 899, com uma tabela hash de 32 008 bytes. A segunda demonstração varia só o lado de BUILD (100 e 100 000 departamentos) com `miniquery_bench`, para mostrar que o BUILD continua na casa dos milissegundos enquanto a tabela hash cresce de 3 208 para 3 200 008 bytes.

14 Como reproduzir

```
make && ./miniquery          # consulta no dataset versionado
make test && make sanitize && make valgrind
make experiments             # gera os datasets (~440 MB) e mede tudo
python3 bench/summarize.py   # medianas, tabelas e graficos
```

O Nested Loop com 100 mil departamentos leva cerca de 21 minutos no total. Para rodar só uma parte: `python3 bench/run_experiments.py -only seletividade`.

15 Conclusão

A etapa 3 acrescenta ao motor materializado do Grupo A o suporte a duas tabelas, o Nested Loop Join, o Hash Join com BUILD na menor relação e PROBE explícitos, o GROUP BY por hash e a instrumentação pedida pelo enunciado. Os dois algoritmos devolvem os mesmos resultados em todos os testes e experimentos. Com Hash Join, a consulta sobre 10 milhões de funcionários levou 2,7 s, contra 19,9 s com Nested Loop, e a diferença chega a 643 vezes quando a relação de BUILD tem 100 mil linhas. Com o JOIN eficiente, o custo restante e a memória vêm da materialização dos intermediários, e não da tabela hash. A validação com Valgrind terminou sem vazamentos e sem erros em todos os programas.

Referências

[1] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–169, 1993.

[2] H. Garcia-Molina, J. D. Ullman e J. Widom. *Database Systems: The Complete Book*. 2. ed. Pearson Prentice Hall, 2008.

[3] Valgrind Developers. *Valgrind User Manual – Memcheck: a memory error detector*. Disponível em: https://valgrind.org/docs/manual/mc-manual.html.